Module 7: Secure Software Supply Chain with SBOMs

UIUC ENG 298 – Spring 2026

Student: Paola Ofcheca

Date: April 27, 2026

Part 1: SBOM Generation Results

Repository Analyzed

The NG911 software repository (github.com/tamu-edu/ng911-dev) was cloned into the GitHub Codespace environment. This repository implements components aligned with NENA/NG911 standards for Next Generation 9-1-1 systems. Two SBOM generation tools were used to catalog all software components and dependencies.

Component Count Comparison

Tool	Format	Components Detected
Syft	SPDX JSON	130
Trivy	CycloneDX JSON	104

SBOM Format Differences

Syft generated an SPDX JSON SBOM with 130 packages, while Trivy produced a CycloneDX JSON SBOM with 104 components. The difference in component count reflects how each tool identifies and catalogs dependencies: Syft performs deeper filesystem-level inspection and includes more indirect dependencies, whereas Trivy focuses primarily on declared dependencies. Format-wise, SPDX uses fields such as 'SPDXID', 'packageVersion', and 'downloadLocation', while CycloneDX uses 'bom-ref', 'purl', and nested 'licenses' blocks — making CycloneDX more compact and structured for machine processing, while SPDX is more verbose and license-focused.

Part 2: Vulnerability Analysis

The Syft SBOM was fed into Grype to identify known CVEs across all detected components. Grype cross-referenced each package against the National Vulnerability Database (NVD) and GitHub Advisory Database (GHSA). A total of 5 vulnerability matches were found.

Top 5 Vulnerabilities

Vulnerability ID	Severity	Component	Version	Fixed In
GHSA-vfmq-68hx-4jfw	High	lxml	5.2.2	6.1.0
GHSA-p423-j2cm-9vmq	Medium	cryptography	46.0.5	46.0.7
GHSA-6w46-j5rx-g56g	Medium	pytest	8.3.3	9.0.3
GHSA-m959-cc7f-wv43	Low	cryptography	46.0.5	46.0.6

GHSA-gc5v-m9x4-r6x2	Medium	requests	2.32.4	2.33.0

CVE Deep Dive: GHSA-vfmq-68hx-4jfw (lxml)

The highest-severity finding is GHSA-vfmq-68hx-4jfw, rated High, affecting lxml version 5.2.2. lxml is a Python library used for processing XML and HTML documents. This vulnerability involves improper handling of malformed or adversarially crafted XML input, which can lead to a heap buffer overflow. An attacker who controls XML input to an application using lxml could trigger memory corruption, potentially resulting in a crash (denial of service) or arbitrary code execution. The vulnerability is resolved in lxml version 6.1.0. Source: https://github.com/advisories/GHSA-vfmq-68hx-4jfw

Part 3: Reflection

How SBOMs Support Security-by-Design

Generating SBOMs for the NG911 repository demonstrates that security-by-design requires complete visibility into every component of a system. The principle of complete mediation (Saltzer and Schroeder, 1975) holds that every access to every resource must be verified. An SBOM extends this idea to the software supply chain: every package and dependency must be known, cataloged, and continuously checked against known vulnerabilities. Without an SBOM, organizations cannot verify what is running in their systems, making it impossible to apply complete mediation at the component level.

Defense in Depth and Least Privilege

The vulnerability findings illustrate why defense in depth is essential. Even though the NG911 repository had only 5 vulnerabilities — one High, three Medium, one Low — a single unpatched High-severity component like lxml could serve as an entry point for an attacker. Combining SBOM generation with vulnerability scanning creates multiple layers of detection, embodying defense in depth. Applying least privilege by limiting which components have network or file access would further reduce the attack surface exposed by these vulnerabilities.

Process Insights

A key insight from this exercise is that different tools produce different results even for the same codebase — Syft detected 130 components while Trivy found 104. This discrepancy highlights that no single tool provides complete visibility, reinforcing the value of using multiple SBOM generators in a CI/CD pipeline. Automating SBOM generation and vulnerability scanning on every code commit ensures that new dependencies are continuously verified, aligning with the secure design lifecycle principle of ongoing assurance rather than point-in-time assessment.